Artificial Intelligence (CS1360)
Intelligent Agents (chap 2)





Outline

- Agents and environments
- Rationality
- PEAS (Performance measure, Environment, Actuators, Sensors)
- Environment types
- Agent types

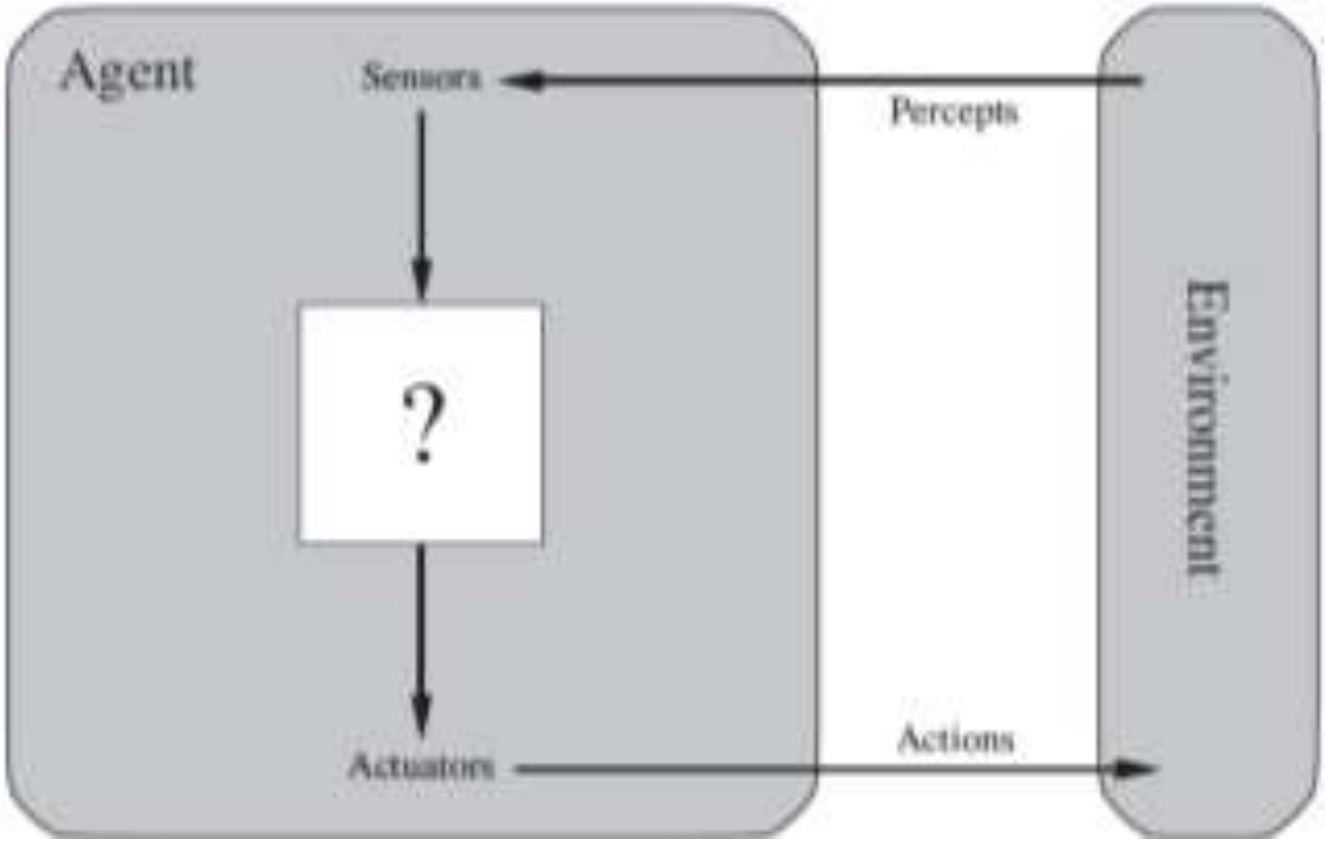
Agents



What must a taxi-driving AI agent sense, decide, and optimize?

Agents

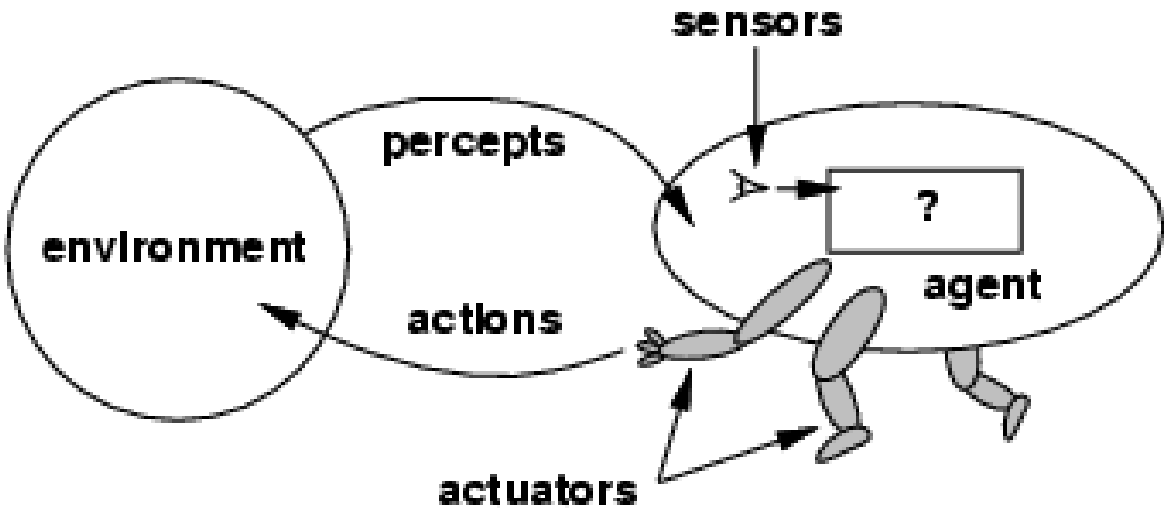
- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.



Agents

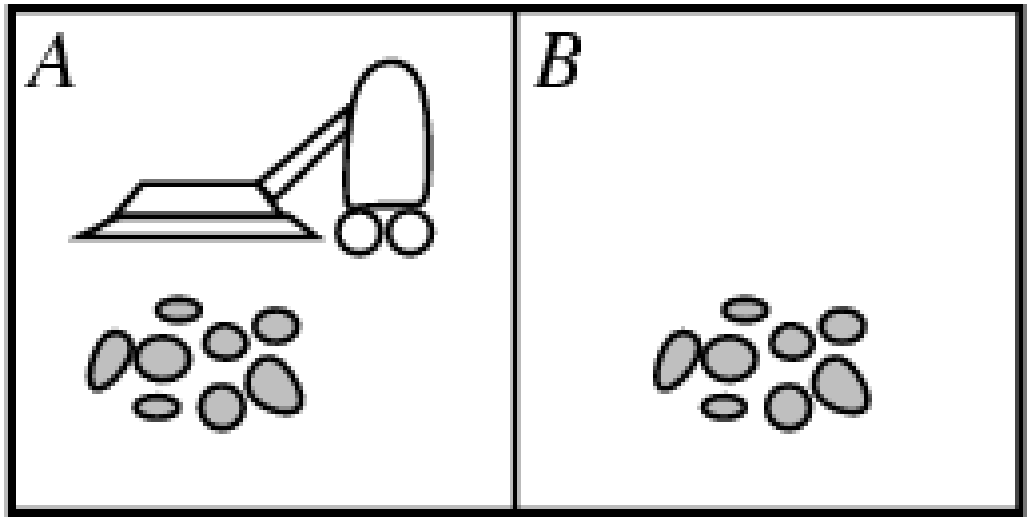
- Human agent:
  - eyes, ears, and other organs for sensors;
  - hands, legs, mouth, and other body parts for actuators
- Robotic agent: cameras and infrared range finders for sensors; various motors for actuators
- Virtual agent: data and input devices for sensors; signals and output devices for actuators.

Agents and environments



- The agent function maps from percept histories to actions:
  $[f: \mathcal{P}^* \rightarrow \mathcal{A}]$
- The agent program runs on the physical system to produce f
- Agent = function + program
- We design programs to approximate rational functions.

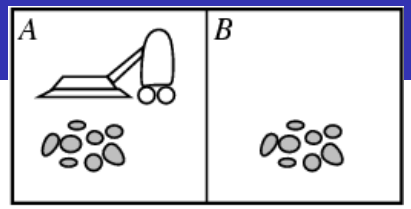
Vacuum-cleaner world



Percepts: location and contents, e.g., [A, Dirty]

Actions: *Left*, *Right*, *Suck*, *NoOp*

A vacuum-cleaner agent



What is the right function?

Can it be implemented in as a small agent program?

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

function REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

if *status* = *Dirty* **then return** *Suck*
else if *location* = *A* **then return** *Right*
else if *location* = *B* **then return** *Left*

Rational agents

- Rational Agent: For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

- Performance measure: An objective criterion for success of an agent's behavior.

  E.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.

Rational agents

- Rationality is distinct from omniscience (all-knowing with infinite knowledge)
    - An omniscient agent knows the actual outcome of its actions and can act accordingly; but omniscience is impossible in reality.

Rational agents

- Agents can perform actions in order to:
  - Explore: obtain useful information (information gathering, exploration)
  - Learn: modify future percepts and actions

- An agent is autonomous if its **behavior** is determined by its own experience (with ability to learn and adapt)
  - A rational agent should be autonomous.
  - a vacuum-cleaning agent that learns to foresee where and when additional dirt will appear will do better than one that does not.

Rational agents

- An intelligent agent can only be rational if we first define what “doing well” means.

- Example: Self-driving car. Drive fast or save fuel?
  - Which matters more?
  - How do we measure them?
  - What tradeoffs are acceptable?

- **Performance measures** guide everything an agent does
- It’s also important to know where the agent operates and the limitations.

Specifying the task environment: PEAS

- PEAS: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors

- Task environment
  - Performance measure
  - Environment
  - Actuators
  - Sensors

PEAS

Agent: Medical diagnosis system

- Performance measure: Healthy patient, minimize costs, lawsuits
- Environment: Patient, hospital, staff
- Actuators: Screen display (questions, tests, diagnoses, treatments, referrals)
- Sensors: Keyboard (entry of symptoms, findings, patient's answers)

PEAS

Agent: Part-picking robot

- Performance measure: Percentage of parts in correct bins
- Environment: Conveyor belt with parts, bins
- Actuators: Jointed arm and hand
- Sensors: Camera, joint angle sensors

PEAS

Agent: Interactive English tutor

- Performance measure: Maximize student's score on test
- Environment: Set of students
- Actuators: Screen display (exercises, suggestions, corrections)
- Sensors: Keyboard

Exercise

Agent type	Performace Measure	Environment	Actuators	Sensors
Robot Soccer Player				
Internet book-shopping agent				
Autonomous mars rover				

Task environment	observable	Deterministic	Episodic	Static	Discrete	Agent
Robot Soccer						
Internet book-shopping agent						
Autonomous mars rover						

Agent type	Sensors	Actuators	Environment	Performace Measure
Robot Soccer Player	Camera, touch sensors, accelerometers, orientation sensors,	Devices (e.g legs) for locomotion and kicking	Field, ball, own team, other team, own body	Winning game, goals for/against
Internet book-shopping agent	Web pages, user request	Follow link, enter/submit data in field, display to user	internet	Obtain request/interesting books, minimize expenditure
Autonomous mars rover	Camera, touch sensors, accelerometers, orientation sensors, wheel, radio receiver	Wheels/legs, sample collection device, analysis devices, radio transmitter	Launch vehicle, mars, lander	Terrain explored and reported, samples gathered and analyzed

Environment

Features:

- Observability
- Determinism
- Episodic vs Sequential
- Static vs Dynamic
- Discrete vs Continuous
- Single vs Multi-agent

Environment types

- **Fully observable vs. partially observable**
  - Fully observable: an agent's sensors give it access to the complete state of the environment at each point in time.
  - Partially observable: noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data.
  - Unobservable: the agent has no sensors at all.

Environment types

- **Deterministic vs. Nondeterministic**
  - Deterministic: actions are predictable.
  - If the next state of the environment is completely determined by the current state and the action executed by the agent, then we say the environment is deterministic; otherwise, it is stochastic.
  - Examples:
    - **Taxi driving** is clearly stochastic in this sense, because one can never predict the behavior of traffic exactly.
    - **The vacuum world** as we described it is deterministic, but variations can include stochastic elements such as randomly appearing dirt and an unreliable suction mechanism.

Environment types

- **Episodic vs. Sequential**
  - Does the next “episode” depend on the actions taken in the previous episodes?
  - Episodic: The agent's experience is divided into atomic "episodes" (each episode consists of the agent perceiving and then performing a single action), and the choice of action in each episode depends only on the episode itself.
  - In sequential environments: the current decision could affect all future decisions.
  - Episodic environments are much simpler than sequential environments because the agent does not need to think ahead.

Environment types

Episodic vs. Sequential

- Examples:
  - Many classification tasks are episodic, such as an agent that has to spot defective parts on an assembly line bases each decision on the current part, regardless of previous decisions; moreover, the current decision doesn't affect whether the next part is defective.
  - Chess and taxi driving are sequential.

Environment types

- **Static vs. Dynamic**
  - If the environment can change while an agent is deliberating, then we say the environment is dynamic for that agent; otherwise, it is static.
  - Semidynamic: If the environment itself does not change with the passage of time but the agent's performance score.
  - Examples:
    - Dynamic: Taxi driving
    - Semidynamic: Chess, when played with a clock
    - Static: Crossword puzzles

Environment types

- Discrete **vs**. Continuous.
  - A limited number of distinct, clearly defined percepts and actions.
  - Examples:
    - Chess has a discrete set of percepts and actions.
    - Taxi driving is a continuous-state and continuous-time problem. Taxi-driving actions and percepts are also continuous.

Environment types

- **Single agent vs. multiagent**
  - Single agent: an agent operating by itself in an environment.
  - Example: an agent solving a crossword puzzle by itself is clearly in a single-agent environment, whereas an agent playing chess is in a two-agent environment.
  - Multiagent environment
    - Competitive : chess
    - Cooperative : taxi
    - Partially cooperative or competitive

Environment types

- The environment type largely determines the agent design
- The hard case is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

	Chess with a clock	Chess without a clock	Taxi driving	Crossword puzzle	Medical diagnosis
Fully observable	Yes	Yes	No	Yes	No
Deterministic	Strategic	Strategic	No	Yes	No
Episodic	No	No	No	No	No
Static	Semi	Yes	No	Yes	No
Discrete	Yes	Yes	No	Yes	No
Single agent	No	No	No	Yes	Yes

Exercise

Task environment	Agents	Discrete	Static	Episodic	Deterministic	observable
Robot Soccer						
Internet book-shopping agent						
Autonomous mars rover						

Exercise

Task environment	Agents	Discrete	Static	Episodic	Deterministic	observable
Robot Soccer	Mutli	Countinuous	Dynamic	Sequential	Stochastic	Partially
Internet book-shopping agent	Single	Discrete	Static	Sequential	Deterministic	Partially
Autonomous mars rover	Single	Countinuous	Dynamic	Sequential	Stochastic	Partially

Part 2

Agent functions and programs

- An agent is completely specified by the agent function mapping percept sequences to actions.

- The agent program implements the agent function mapping the percepts to actions.

Agent=architecture + program.

Assume: Architecture= sort of computing device with physical sensors and actuators.

- Aim of AI is to design the agent program: find a way to implement the rational agent function concisely.

Table-lookup agent

```
Function Table-Driven-Agent(percept)
Static: percepts, a sequence, initially empty
        table, a table of actions, indexed by percept
               sequences, initially fully specified

  append percept to the end of percepts
  action <- Lookup(percepts, table)

Return action
```

The table agent program is invoked for each new percept and returns an action each time. It keeps track of percept sequences using its own private data structure.

Table-lookup agent

- Drawbacks:
  - Huge table
  - Take a long time to build the table
  - No autonomy
  - Even with learning, need a long time to learn the table entries.

- Example : let P be the set of possible percepts and T be the lifetime of the agent (the total number of percepts it will receive) then the lookup table will contain $\sum |P|^t$ (t=0…T) entries.
- The table of the vacuum agent (VA) will contain more than 4^T entries (VA has 4 possible percepts).

Table-lookup agent

- The key challenge for AI is to find out how to write programs that, to the extent possible, produce rational behavior from a smallish program rather than from a vast table.

Agent types

- Four basic types that embody the principles underlying almost all intelligent systems:
  - Simple reflex agents
  - Model-based reflex agents
  - Goal-based agents
  - Utility-based agents

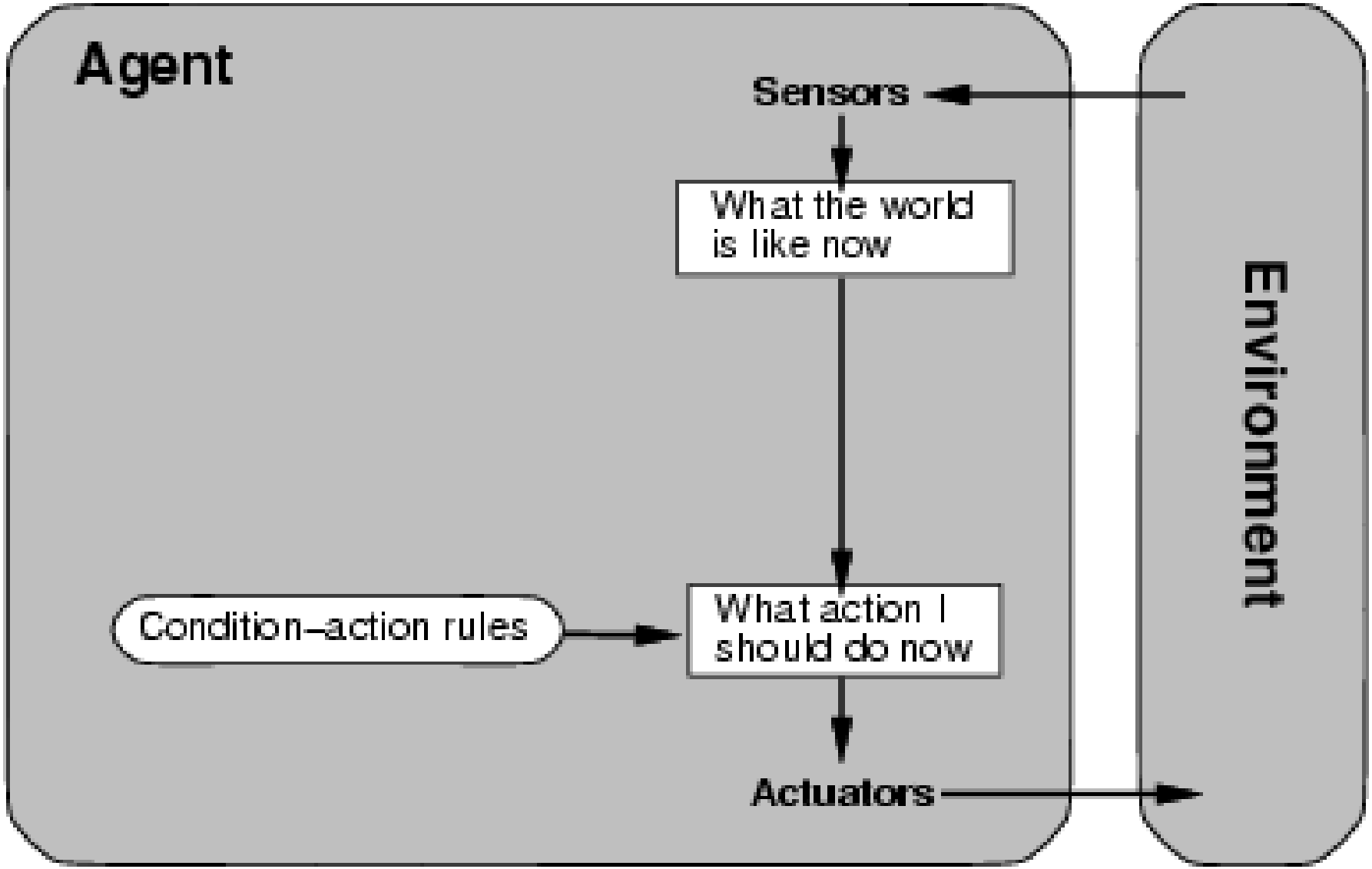
  - and then, there is the learning agent!

Simple reflex agents Program

Single current percept: the agent select an action on the basis current percept, ignoring the rest of percept history.

Example: The vacuum agent (VA) is a simple reflex agent, because its decision is based **only** on the current location and on whether that contains dirt.

Schematic diagram of a simple reflex agents



Agent program for a vacuum-cleaner agent

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

- The vacuum agent program is very small compared to the corresponding table : it cuts down the number of possibilities from 4^T to 3. This reduction comes from the ignoring of the history percepts.

Simple reflex agents Program

- Rules relate
  - “State” based on percept
  - “Action” for agent to perform
  - “Condition-action” rule:
    - If a then b: e.g.
      - vacuum agent (VA) : if in(A) and dirty(A), then vacuum
      - taxi driving agent (TA): if car-in-front-is-braking then initiate-braking.

Simple reflex agents program

```
Function Simple-Reflex-Agent(percept)
Static: rules, set of condition-actions
rules;

          state  <- Interpret-Input(percept)
          Rule   <- Rule-Match(state, rules)
          action <- Rule-Action[Rule]

Return action
```

A **simple reflex agent** acts according to rule whose condition matches the current state, as defined by the percept.

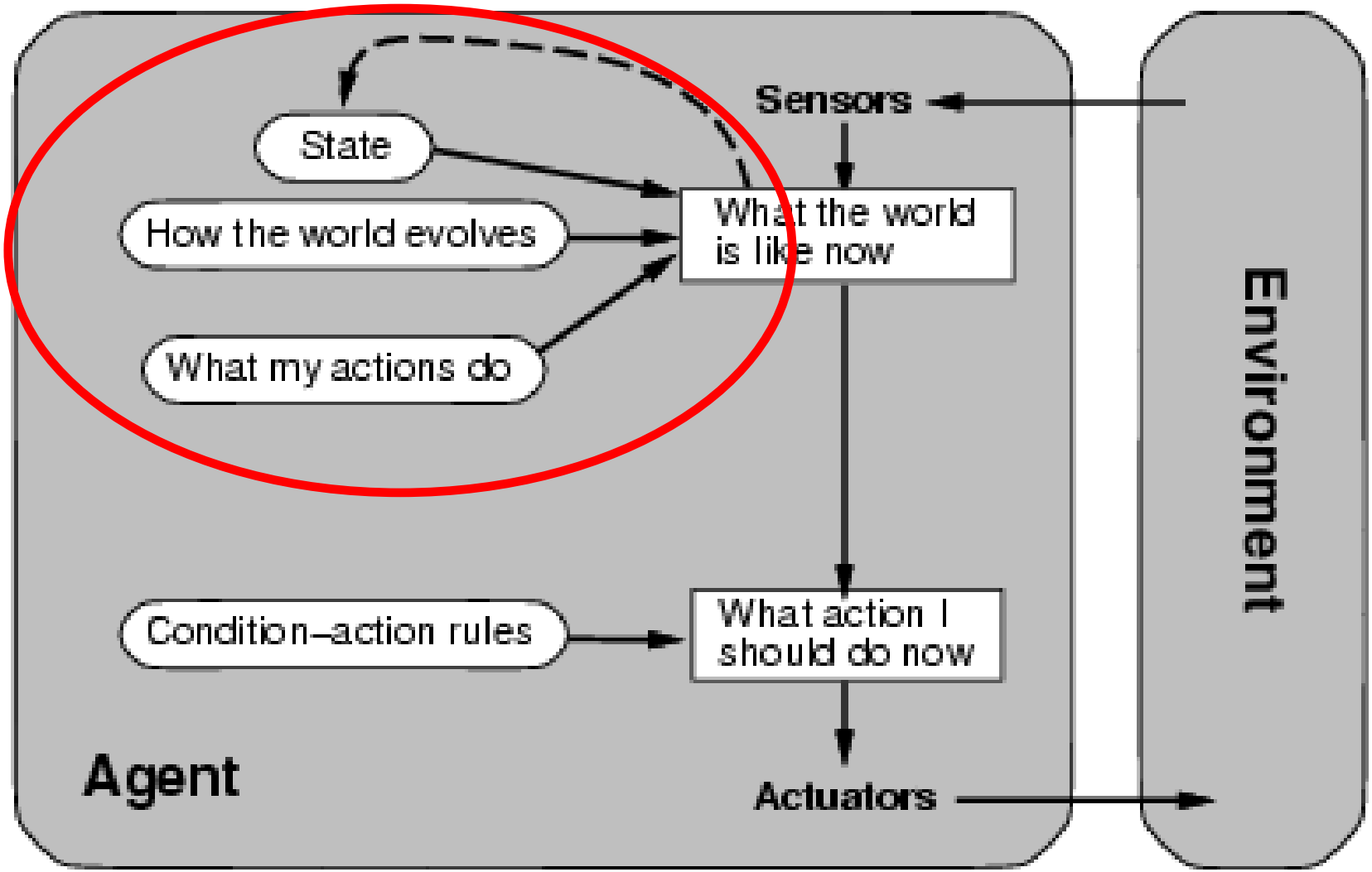
Simple reflex agents program

- Simple, but VERY limited
  - Must be fully observable to be accurate
    - Limited intelligence (decision can be made only if the environment is fully observable)
  - Example : vacuum agent without location sensor, and has only a dirt sensor (2 possible percepts : [dirty] and [clean]) :
    - The action for [dirty] is suck.
    - What is the action for [clean] ? Moving left fails forever if it happens to start in location A.

Model-based reflex agents

- Solution to partial observability problems
  - Maintain state
    - Keep track of parts of the world can't see now
    - Maintain internal state that depends on the percept history
- Update previous state based on
  - Knowledge of how world changes,
    - e.g. TA : an overtaking car generally will be closer behind than it was a moment ago.
  - Knowledge of effects of own actions,
    - e.g. TA: When the agent turns the steering wheel clockwise the car turns to the right.
  - knowledge about "how the world works"—whether implemented in simple Boolean circuits or in complete scientific theories—is called a "model of the world".
- An agent that uses such a model is called a model-based agent.

Schematic diagram of a Model-based reflex agents



Model-based reflex agents

```
Function Model-based-Reflex-Agent(percept)
Static: state, a description of the current world state
        rules, set of condition-actions rules;
        actions, the most recent action, initially none

  State<-Update-State(oldInternalState,LastAction,percept)
  rule<- Rule-Match(State, rules)
  action <- Rule-Action[rule]

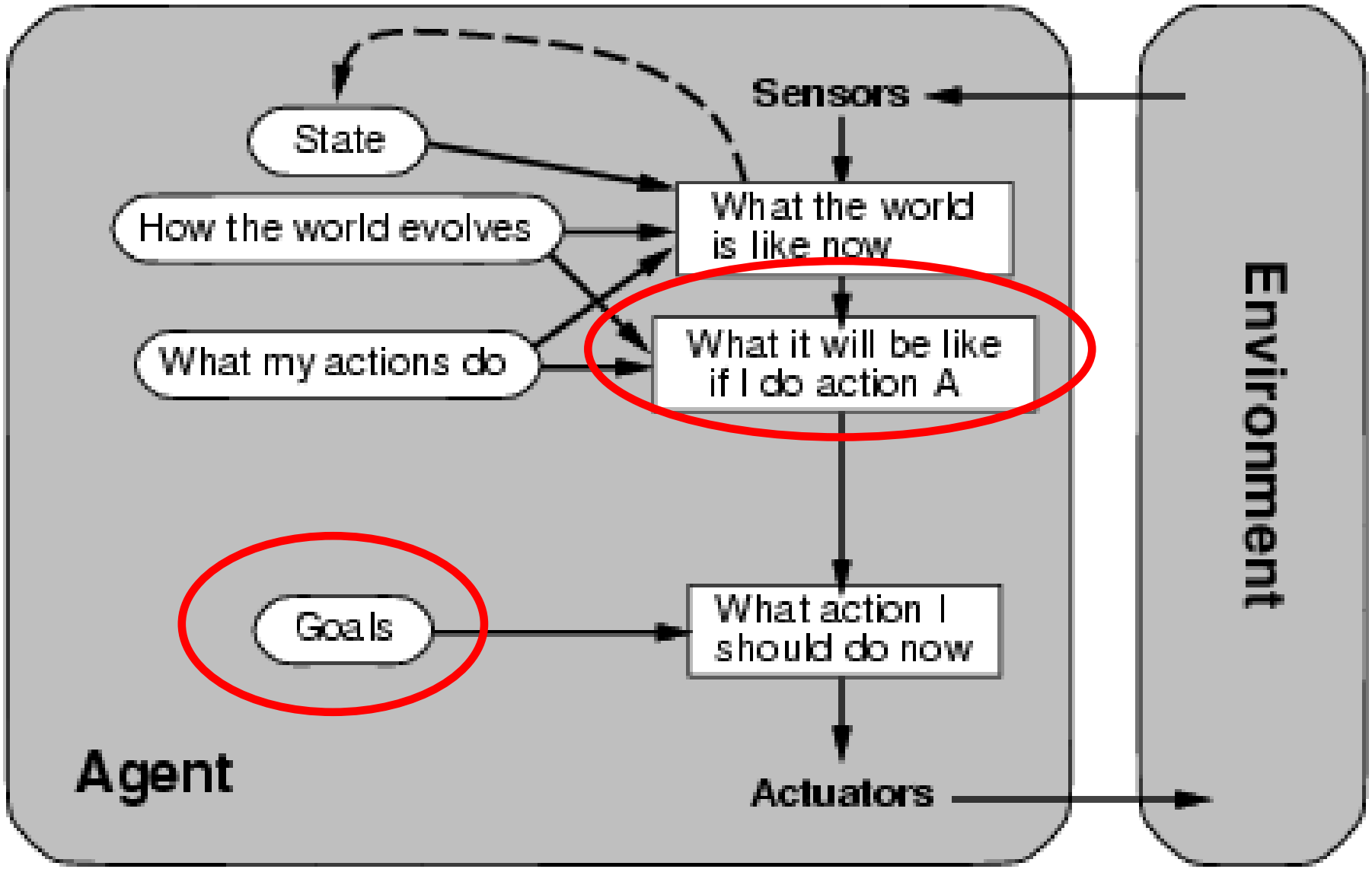
Return action
```

A model-based reflex agent. It keep track of the current state of the world using an internal model. It then chooses an action in the same way as the reflect agent

Goal-based agents

- The current state of the environment is not always enough to decide what to do.
  - The taxi can turn left, right or go straight on. The decision depends on where the taxi is trying to get to (for example : being at the passenger's destination).

Goal-based agents



A model-based, goal-based agent

```
Function Model-based-Reflex-Agent(percept)
Static: state, a description of the current world state
        rules, set of condition-actions rules;
        actions, the most recent action, initially none

State<-Update-State(oldInternalState,LastAction,percept)
rule   <- Rule-Match(State, rules, goal)
action  <- Rule-Action[rule]

Return action
```

A goal-based agent keeps track of the current state of the world using an internal model as well as a set of goals it is trying to achieve. It then chooses an action that leads to the achievement of the goal.

Utility-based agents

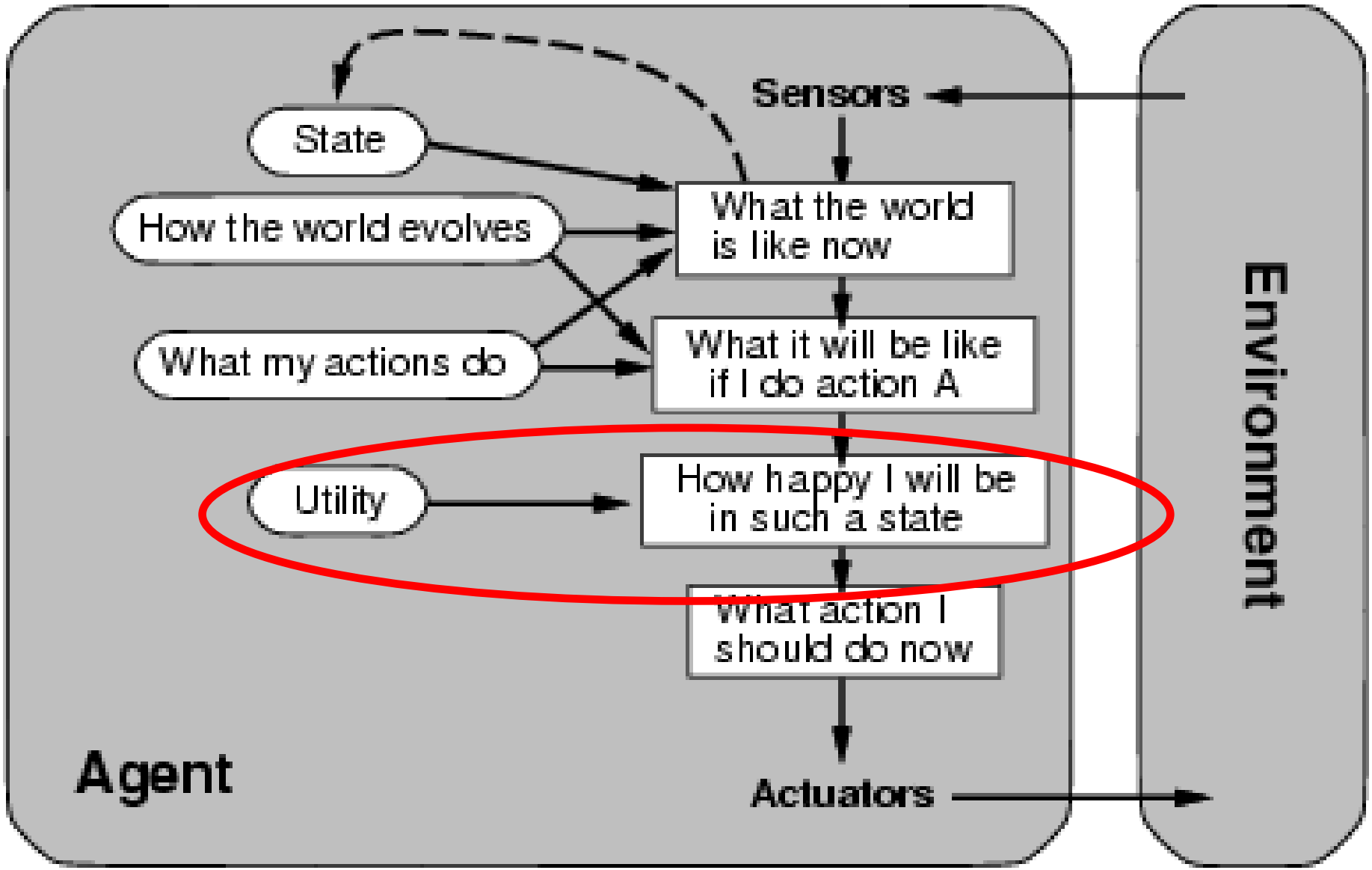
- Goal:
  - Goals alone are not enough to generate high-quality behavior in most environments.
  - Issue: only binary: achieved/not achieved
  - Goals just provide a crude distinction between "happy" and not "happy" states, whereas more general performance measure should allow a comparison of different world sequences.
  - Want more nuanced:
    - Not just achieve state, but faster, cheaper, smoother,...

Utility-based agents

- Solution: Utility
  - Utility function: state (sequence percepts) -> value
  - Select among multiple or conflicting goals.
  - Utility is a function that maps a state onto real number, which describes the associated degree of happiness
    - helps to make decision when there are conflicting goals (like speed and safety)

```
The right decision = Function (percept, goal)
  + Quicker + Safer + Reliable + Less cost
```

Utility-based agents



A model-based, utility-based agent

```
Function Model-based-utility-based -Agent(percept)
Static: state, a description of the current world state
                rules, set of condition-actions rules;
                actions, the most recent action, initially
  none

  State<-Update-State(oldInternalState,LastAction,percept)
  AllRules    <- Rule-Match(State, rules, goal)
  bestRule    <- utility(AllRules)
  action       <- Rule-Action[bestRule]

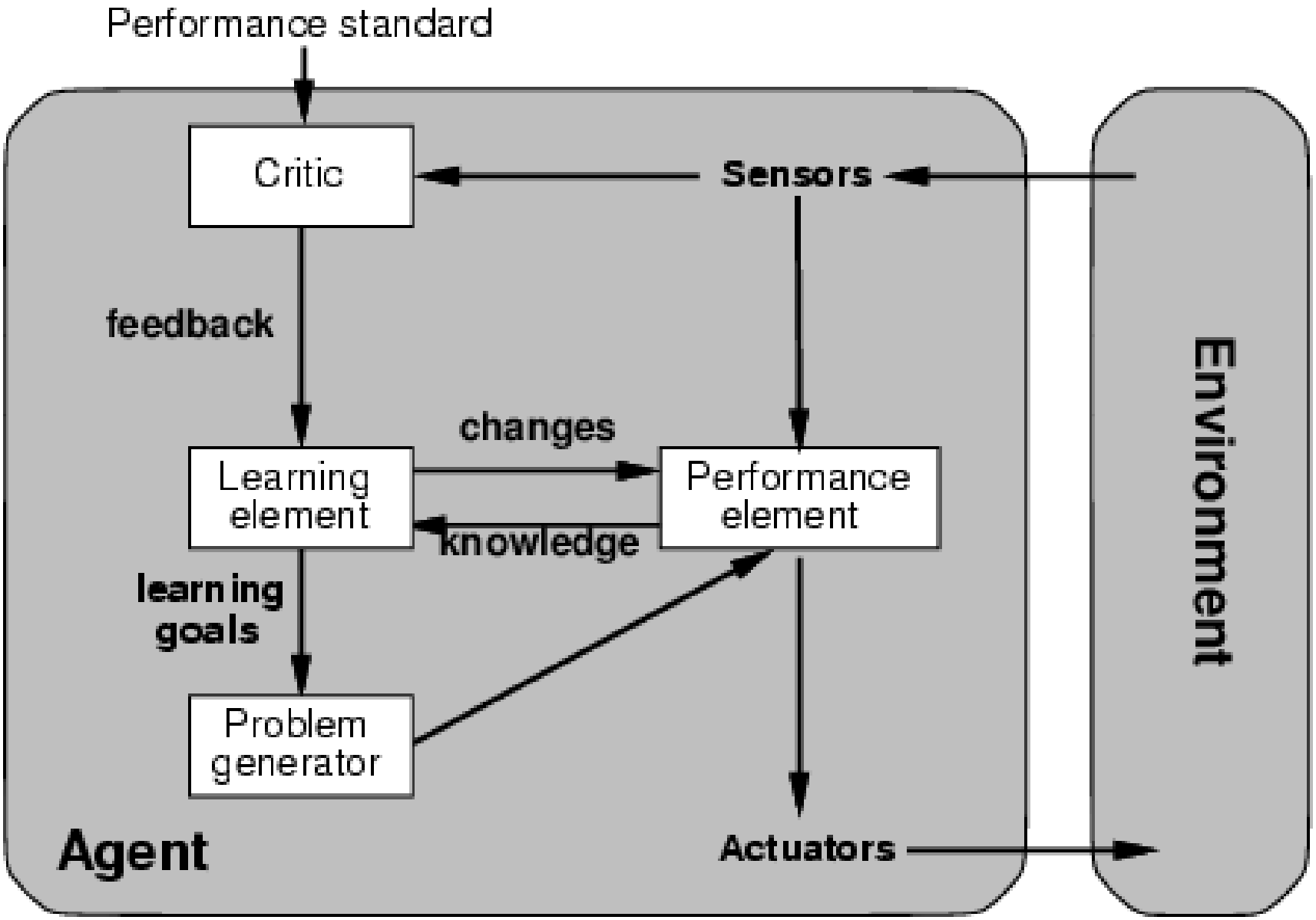
Return action
```

A utility-based agent uses a model of the world along with utility function that measures its preferences among states of the world.

Learning agents

- All agents can improve their performance through learning.
  - Learning: allow agent to match new states/actions
  - A learning agent can be divided into four conceptual components:
    - Learning element: responsible for making improvements
    - Performance element: selects external actions based on percept (entire agent in previous cases). The performance element is what we have previously considered to be the entire agent: it takes in percepts and decides on actions.
    - Critic: gives feedback to learning about success (it tells the learning element how well the agent is doing with respect to a fixed performance standard.
    - Problem generator: suggests actions to find new states.

Learning agents



Example : Learning agents - Taxi driving

- The performance element consists of whatever collection of knowledge and procedures the TA has for selecting its driving actions.
- The critic observes the world and passes information along to the learning element.
  - For example, after the taxi makes a quick left turn across three lanes the critic observes the shocking language used by other drivers.
  - From this experience the learning element is able to formulate a rule saying this was a bad action, and the performance element is modified by installing this new rule.

Example : Learning agents : Taxi driving

- The problem generator may identify certain areas of behavior in need of improvement and suggest experiments
  - such as testing the brakes on different road surfaces under different conditions.
- The learning element can make change in any Knowledge of previous agent types
  - observation between two states (how the world evolves), observation of results of actions (what my action do).

Agent Design Leads to AI Techniques

- Goals, Utility -> Search
- Multiagent -> Adversarial search
- Constraints -> CSP
- Uncertainty ->Bays
- Learning -> ML

Summary

- Agents interact with the environment: They perceive and act.
- Rationality is driven by task, goal and limitations
- The environment and goal determines the design
- The design determines the AI technique.